



Paradigmas Programación

Profr. M.C. Jesus H. Abundis Patiño
jesus.abundis@info.uas.edu.mx

Facultad de Informática, Culiacán
Universidad Autónoma de Sinaloa

Resumen

- **Cinco unidades**

1. Introducción a los paradigmas de programación
2. Fundamentos de la Programación Orientada a Objetos (POO) y Lenguajes de Programación
3. Conceptos clave de la Programación Orientada a Objetos (POO)
4. Fundamentos de la Programación Orientada a Eventos
5. Aplicación Práctica: Creación y Manipulación de Formularios

Requisitos

- **80% asistencias**
- **Proyecto final**
- **Exámenes**

Evaluación

- Asistencias: **10%**
- Exámenes parciales: **60%**
- Tareas: **15%**
- Proyecto final: **15%**

Lenguajes

- Orientado a objetos: **Python**
- Orientado a eventos: **JavaScript**

1. Introducción a los paradigmas de programación

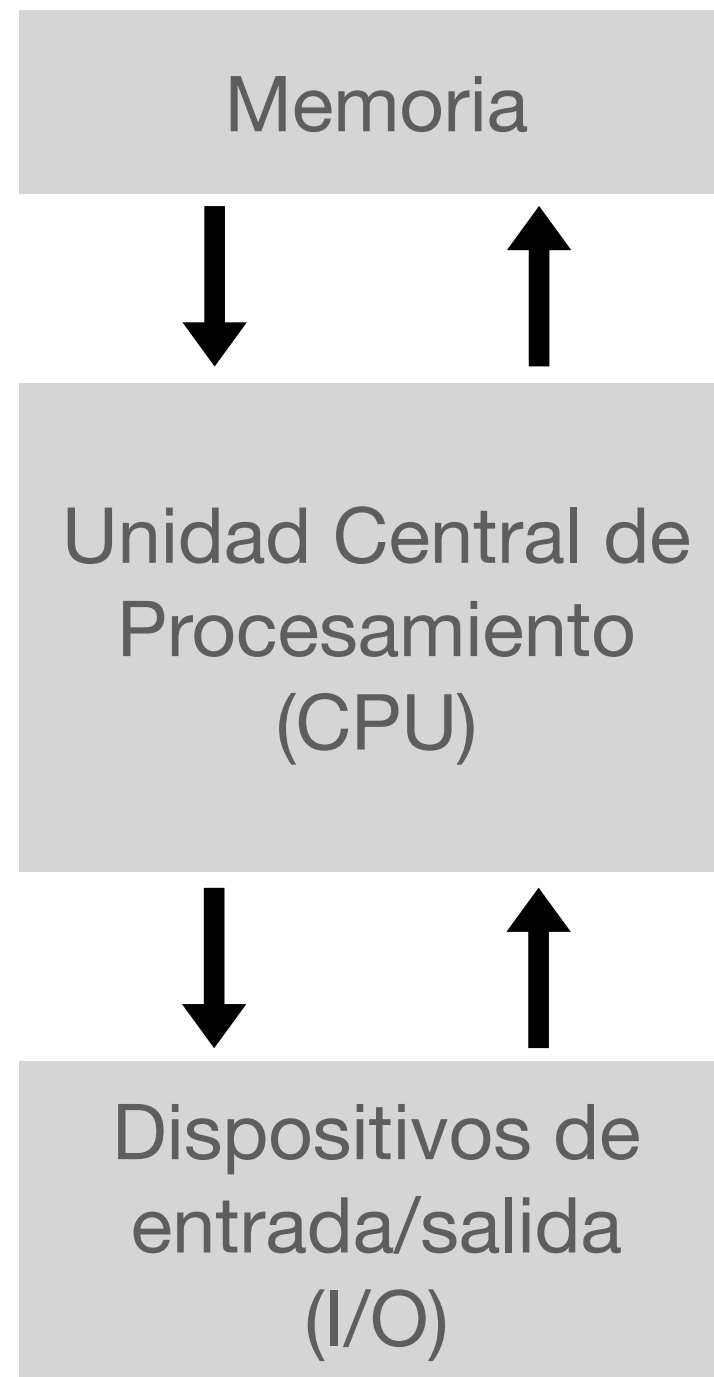
Temática

- **Conceptos básicos**
- Clasificación por niveles
- Clasificación por paradigmas
- Paradigmas emergentes

Conceptos básicos

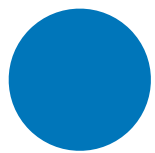


Computador





Bit





Conceptos

Poniendo en perspectiva

¿Qué distancia viaja la luz en el tiempo que mi procesador de 4GHz realiza una operación?

Respuesta: ¡7.5cm en promedio!

Detengamonos a contemplar esta cantidad. ¡La luz viaja desde la Tierra a la Luna en 1.3 segundos! Es extremadamente rápida y aún así solo alcanza a viajar la longitud de mi mano cuando mi procesador realiza una operación.



Conceptos

Instrucciones al CPU

$$h = g / 2 * t^{**2}$$

1. Lee la dirección de memoria t
2. Eleva al cuadrado y ponlo en dirección temporal
3. Lee la dirección g
4. Divide entre dos
5. Multiplica con número en memoria
6. Almacena en dirección h



Conceptos

Lenguajes de programación

Es un conjunto de instrucciones, reglas y sintaxis que le permiten al ser humano comunicarse con la computadora.

Así como en los lenguajes hablados, cada lenguaje tiene su propia sintaxis, semántica, estructura, vocabulario e incluso abreviaciones.

- Más fácil de entender que las instrucciones que recibe el CPU
- Se debe traducir para que el CPU lo entienda



Conceptos

Principios para el diseño de un lenguaje

1. Sintaxis.
2. Nombres y Tipos.
3. Semántica.



Conceptos

Principios para el diseño de un lenguaje

1. Sintaxis

- La sintaxis describe lo que constituye un programa estructuralmente correcto.
- Nos dice el conjunto de palabras y símbolos que el programador puede utilizar. También nos da la gramática, es decir, el orden y combinación de palabras y símbolos que debemos emplear.
- Nos dice cómo debe escribirse el código (reglas estructurales)
- Estas reglas son las que el compilador o intérprete usa para comprender y ejecutar nuestro programa.

Ejemplo en C:

```
int length = 5; // sintaxis correcta
```

```
int length 5; // errores (¿Cuáles?)
```



Conceptos

2. Nombres y Tipos

- Todo lenguaje incluye un conjunto de reglas bien diseñado para nombrar entidades; variables, funciones, clases, parámetros y demás.
- Los nombres también contienen otras propiedades: alcance (scope), visibilidad y vinculación (binding).



Conceptos

2. Nombres y Tipos

Alcance:

Región específica de un programa dónde una entidad nombrada (variable, función, clase) es visible y accesible. Determina cuando la vinculación de una variable es válida y nos ayuda a prevenir colisión de nombres.

Visibilidad:

Define si se puede acceder a una entidad desde fuera de la clase donde fue definida (public, private).

Vinculación:

Asociación de un identificador (variable, función, clase) con una entidad de datos específica, ubicación en memoria, o valor. Nos permite referenciar y manipular memoria o datos.



Conceptos

Alcance

Local: variables definidas dentro de funciones

```
import math
alpha = 2.5
beta = 5
def distance(x, y):
    length = math.sqrt(x**2 + y **2)
    return length
def increment(alpha):
    alpha += 1
    return alpha
def double():
    global beta
    beta *= 2
```

```
print( distance(3, 4) )
print( increment(alpha) )
print( double() )
print( 'alpha = ', alpha )
print( 'beta = ', beta )
print( 'length = ', length )
```

5

3.5

None

alpha = 2.5

beta = 10

NameError

Global: variables definidas fuera de una función.

Solo visibles (accesibles) dentro de la función.

17

Accesibles desde cualquier lugar del archivo (módulo en python).



Conceptos

Visibilidad en Java

public: accesible desde cualquier lugar del programa

```
public class Estudiante {  
    // variable privada  
    private int age;  
    // variable pública  
    public String name;  
  
    // method publico  
    public void setAge(int newAge) {  
        if (newAge > 0) {  
            age = newAge;  
        } else {  
            System.out.println("La edad no puede ser negativa o cero");  
        }  
    }  
  
    // meted público  
    public int getAge() {  
        return age;  
    }  
}
```

private: accesible solo desde la clase donde es definido

```
public class Ejemplo {  
    public static void main(String[] args) {  
        Estudiante person = new Estudiante();  
  
        // Acceso a variable pública (permitido)  
        person.name = "Julia";  
  
        // Accediendo a variable privada (error durante  
        compilación)  
        // person.age = 25;  
  
        // Accediendo y modificando variable privada a través de  
        método público (recomendado)  
        person.setAge(25);  
        System.out.println(person.name + " tiene " +  
        person.getAge() + " años de edad");  
  
        person.setAge(-1); // Imprimirá mensaje indicando que  
        esto no es posible  
    }  
}
```



Conceptos

Visibilidad en Python

En python no se fuerza que las variables tengan un tipo público o privado. Sin embargo, existe una convención que los programadores suelen utilizar para decirles a otros cuando desean que una variable sea pública o privada:

- **Cualquier variable en una clase que no empiece con un guión bajo se considera pública:** Pueden accederse y modificarse desde cualquier lugar
- **Cuando la variable empieza con el guión bajo le estas diciendo a los programadores que dicha variable no debe accederse ni modificarse** fuera de la clase donde se definió (parte de una implementación interna). Sin embargo, el intérprete no lo fuerza y podemos acceder o modificar dichas variables.



Conceptos

Visibilidad en Python

```
class Estudiante:
    def __init__(self):
        self.carrera = 'Informatica' # variable pública

estudiante = Estudiante()
print(estudiante.carrera) # accesible
estudiante.carrera = "ciencia de datos" # modificable
```

```
class Estudiante:
    def __init__(self):
        self._ID = '12345' # variable que se desea privada

estudiante = Estudiante()
print(estudiante._ID) # accesible pero no se recomienda
```



Conceptos

Algunas reglas para asignar nombres en Python

Reglas obligatorias

- Caracteres permitidos: Letras de la “a-z”, “A-Z”, dígitos “0-9” y el Guion bajo “_”.
- Sensible a mayúsculas: **name, Name, NAME, son distintos nombres.**
- Sin caracteres especiales ni espacios: No están permitidos caracteres fuera de los anteriores, es decir, @, #, &, etc. no son válidos. **Se recomienda usar guion bajo cuando un nombre contiene múltiples palabras.**
- No debe comenzar con un dígito: **Un nombre válido comienza con una letra o un guion bajo.**
- No se permiten las palabras reservadas: **No puedes emplear palabras como for, if, else, class, etc.**



Conceptos

Recomendaciones: PEP 8 (Python Enhancement Proposal) (Guía de estilos oficial de Python)

Variables, funciones y métodos	Minúsculas con guiones bajos (snake_case), (descriptivas, concisas, sustantivos, sin ambigüedad)	dipolo_electrico, obtener_costo().
Clases y excepciones	Mayúscula inicial para cada palabra (CapWords)	HttpRequest ModuleNotFoundError
Modulos y paquetes	Minúsculas; una sola palabra de ser posible o, en caso contrario, usar guión bajo.	geometry.py db_connect.py
Constantes (globales)	Mayúsculas con guiones bajos. (Usar nombres descriptivos y con significado)	PI DB_URL



Conceptos

Leer recomendaciones de PEP 8 y poner en práctica

<https://peps.python.org/pep-0008/>

Realizar resumen de la sección de nombres de PEP 8



3. Semántica

- ## Ejemplos en C:

Ejemplos en Python:

24



Conceptos

Conceptos fundamentales en semántica

Python

Tipos dinámicos
(Dynamic Typing)

- El intérprete infiere el tipo de una variable durante la ejecución basado en el valor que esta toma.
- La variable puede reasignarse a un valor de distinto tipo sin ocasionar un error de compilación.
- Semántica: El nombre de la variable es una referencia que puede apuntar a cualquier tipo de objeto.

Java

Tipos estáticos
(Static Typing)

- El tipo de la variable se debe declarar antes de poder utilizarla
- Su tipo no puede cambiar.
- El compilador hace un chequeo de esto durante la compilación.
- Semántica: La variable es un contenedor fijo para un tipo específico de datos fijado por el compilador.



Conceptos

Python

Tipos dinámicos (Dynamic Typing)

```
# Se asigna un valor de tipo 'int' a 'variable'
variable = 10
print(f"Valor: {variable}, Tipo: {type(variable)}")
```

```
# La variable se reasigna a un 'str'
variable = "Hello, world!"
print(f"Valor: {variable}, Tipo: {type(variable)}")
```

```
# Y ahora a un float
variable = 3.14
print(f"Valor: {variable}, Tipo: {type(variable)}")
```

Java

Tipos estáticos (Static Typing)

```
public class StaticTypingExample {
    public static void main(String[] args) {
        int variable = 10; // Declarada como entero

        // error de compilación:
        // variable = "hello"; // Tipo incompatible
    }
}
```



Conceptos

Type hints (pistas de tipo) en Python (PEP 484)

Debido a que en Python no se fuerza el tipo para una variable, el programador puede utilizar anotaciones conocidas como **type hints** que indican el tipo de datos esperado de variables, argumentos de funciones y valores de regreso.

Existen programas conocidos como **verificadores de tipos estáticos**, los cuales podemos ejecutar antes de correr nuestro programa, que checan el tipo de aquellas variables que cuentan con un **type hint** y nos indican si podemos tener error en nuestro código. Dos muy conocidos son **mypy** y **pyright**, aunque los ambientes de desarrollo integrados (IDE) modernos ya cuentan con uno propio.

```
height: float = 1.79
```

```
def presentar(name: str, age: int) -> str:  
    return f"{name} tiene {age} años"
```

```
edades: list[int] = [19, 21, 20]
```

```
calificacion: dict[str, float] = {'Pedro': 8.7, 'Ana': 9.1}
```

Además de ayudarnos a detectar errores temprano, estas pistas sirven como documentación, permitiéndonos entender mejor el código y también nos ayudan a mantener una estructura clara de este.



Conceptos

Python utiliza tipos fuertes (strong typing)

Python no permite operaciones en dónde los datos no cuentan con el tipo esperado, es decir, aquellas que no estén bien definidas, arrojando en esos casos el error **TypeError**.

Ejemplos:

- Añadir un número con un string: `8 + 'hello'` **TypeError**
- Tipo no esperado en una función: `len(13)` **TypeError**
- Problemas con listas:
`[5, 2, 6] > 0` **TypeError**
- Operaciones booleanas con valores inesperados son truculentas:
`5 + True = 6` `'False' and True = True`
`[1, 4] > [2, 3] = False` `[2, 3] > [2, 3] = False`
`[3, 1] > [2, 3] = True` `[2, 4] > [2, 3] = True`



Conceptos

Todo es un objeto en Python

Las clases, funciones e incluso los enteros son objetos en Python; estos tienen atributos y métodos.

```
type(10) = <class 'int'>
```

```
(10).bit_length() = 4 (número de bits en binario)
```

```
(10).bit_count() = 2 (número de 1's en binario)
```

Los operadores básicos y algunas funciones se encuentran implementadas como métodos de la instancia:

```
(-8).__abs__() = 8
```

```
(5).__add__(-3) = 2
```

(Los métodos que empiezan con doble guión bajo se conocen como **dunder methods**)

Semántica. Todo es un objeto en Python.



Conceptos

Podemos ver todos los tributos y métodos de una instancia o de una clase utilizando el comando **dir**

Clase 'int'

```
>>> dir(int)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__',
 '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattr__', '__getnewargs__', '__getstate__', '__gt__',
 '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__', '__lt__', '__mod__',
 '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__
__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__', '__rrshift_
__', '__rshift__', '__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '
__truediv__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_count', 'bit_length', 'conjugate', 'denominator', 'from_bytes', 'im
ag', 'is_integer', 'numerator', 'real', 'to_bytes']
>>> int.__mul__(5, 3)
15
>>>
```

Función 'my_power'

```
>>> def my_power(base, power):
...     """Return 'base' raised to 'power'"""
...     return base ** power
...
>>> my_power.__name__
'my_power'
>>> my_power.__doc__
'Return 'base' raised to 'power''
>>>
```

```
>>> dir(my_power)
['__annotations__', '__builtins__', '__call__', '__class__', '__closure_
__', '__code__', '__defaults__', '__delattr__', '__dict__', '__dir__', '__
__doc__', '__eq__', '__format__', '__ge__', '__get__', '__getattr__', '__
__getstate__', '__globals__', '__gt__', '__hash__', '__init__', '__in
it_subclass__', '__kwdefaults__', '__le__', '__lt__', '__module__', '__n
ame__', '__ne__', '__new__', '__qualname__', '__reduce__', '__reduce_ex_
__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook_
__', '__type_params__']
>>>
```



Conceptos

Tipos Mutables vs Inmutables

Una distinción semántica crítica es si el valor de un objeto puede cambiar después de su creación.

Tipos mutables:

Listas: `[5, 3.7, 'hola', True]`

Diccionarios: `{'nombre': 'Katia', 'age': 25}`

Sets: `{ 'Maria', 'Luis', 'Karen', 5 }`

Tipos inmutables:

Tuplas: `(3.1, 'adios', True)`

Strings: `"informatics"`

Ints, floats, complex, bool: `5, 2.718, 3+4j, True`

Nota: Tratar de modificar un elemento de una tupla resulta en un error de semántica, es decir, el intérprete arrojará un **TypeError**.



Conceptos

Manejo de errores semánticos en Python

Aún después de escribir un código sin errores de sintaxis es posible tener errores de semántica, tal como la división por cero.

Para manipular estos errores durante la ejecución sin que el programa se detenga, contamos con la estructura **try-except**.

Ejemplo:

```
In [1]: import math
...: def my_sqrt(x):
...:     try:
...:         return math.sqrt(x)
...:     except:
...:         print('Try again with x >= 0')
...:

In [2]: my_sqrt(9)
Out[2]: 3.0

In [3]: my_sqrt(-16)
Try again with x >= 0

In [4]: █
```



Conceptos

Sintaxis y semántica de python oficial

The screenshot shows the Python Language Reference documentation page for version 3.14.2. The page is titled "The Python Language Reference" and provides an overview of the manual's content. It includes a table of contents on the left and a detailed description of the manual's scope on the right.

Previous topic
8. Editors and IDEs

Next topic
1. Introduction

This page
Report a bug
Show source

The Python Language Reference

This reference manual describes the syntax and "core semantics" of the language. It is terse, but attempts to be exact and complete. The semantics of non-essential built-in object types and of the built-in functions and modules are described in [The Python Standard Library](#). For an informal introduction to the language, see [The Python Tutorial](#). For C or C++ programmers, two additional manuals exist: [Extending and Embedding the Python Interpreter](#) describes the high-level picture of how to write a Python extension module, and the [Python/C API Reference Manual](#) describes the interfaces available to C/C++ programmers in detail.

- 1. Introduction
 - 1.1. Alternate Implementations
 - 1.2. Notation
- 2. Lexical analysis
 - 2.1. Line structure
 - 2.2. Other tokens
 - 2.3. Names (identifiers and keywords)
 - 2.4. Literals
 - 2.5. String and Bytes literals
 - 2.6. Numeric literals
 - 2.7. Operators and delimiters
- 3. Data model
 - 3.1. Objects, values and types
 - 3.2. The standard type hierarchy
 - 3.3. Special method names
 - 3.4. Coroutines
- 4. Execution model
 - 4.1. Structure of a program
 - 4.2. Naming and binding
 - 4.3. Exceptions
 - 4.4. Runtime Components
- 5. The import system
 - 5.1. `importlib`
 - 5.2. Packages
 - 5.3. Searching
 - 5.4. Loading



Conceptos

Tarea

Realizar un mapa conceptual de la historia de los lenguajes de programación en cartulina en equipos de cuatro personas, además de hacer una copia individual en tamaño carta.



Conceptos

Objetivos en el diseño de lenguajes de programación

1. Simplicidad y legibilidad
2. Claridad con los enlaces
3. Fiabilidad
4. Soporte
5. Abstracción
6. Ortogonalidad
7. Implementación eficiente



Conceptos

1. Simplicidad y legibilidad

Los programas deben ser fáciles de escribir, comprensibles y de fácil lectura para un programador promedio. Además, idealmente, deberán ser fáciles de aprender y de enseñar.

Basic: pequeño conjunto de instrucciones

Ruby: sintaxis legible e intuitiva que se asemeja al lenguaje cotidiano.

Python: sintaxis similar al inglés, indentación en lugar de llaves.
Se enfoca en la legibilidad.



Conceptos

2. Claridad con los enlaces

Los elementos en un lenguaje se enlazan con una propiedad. Un buen diseño debe ser claro con respecto al tiempo en que ocurren los enlaces.

Principales tiempos de enlace:

- **Tiempo de la definición del lenguaje:** cuando se define el lenguaje, los tipos de datos básicos se enlazan con etiquetas especiales, llamadas **etiquetas reservadas**, que los representan. En **C** los enteros se ligan al identificador **int** y los número reales a **float**.



Conceptos

- **Tiempo de la implementación del lenguaje:** Cuando el compilador o intérprete del lenguaje es escrito, se ligan valores a representaciones de la máquina. Por ejemplo, el tamaño de un **int** en **C** se determina en el momento de implementación del lenguaje.

El tamaño en memoria de cada tipo de dato depende de la arquitectura de la computadora (32bits vs 64bits) así como del compilador empleado.



Conceptos

Valores típicos para **C**:

char -> 1 byte

short -> 2 bytes

int -> 4 bytes (2 bytes para embebidos)

long int -> 4 bytes (32bits), 8 bytes (64bits)

long long int -> al menos 8 bytes

float -> 4 bytes

double -> 8 bytes

En **C** debemos emplear la función **sizeof** para asegurarnos de los tamaños, por ejemplo, **sizeof(float)**. El estándar de **C** solo garantiza tamaños mínimos y relativos.

Investigar el tamaño de los datos en **Java** y en **Python**.



Conceptos

- **Tiempo de escritura del programa:** En algunos lenguajes, los nombres de las variables se enlazan con tipos, los cuales permanecen asociados con esos nombres durante todo el curso del programa. Una variable puede enlazarse con su tipo durante la declaración: `int x;`
- **Tiempo de compilación:** Las declaraciones y expresiones de un programa se ligan a secuencias de instrucciones equivalentes en el lenguaje de la máquina al momento de la compilación.
- **Tiempo de carga del programa:** Cuando cargamos el código de máquina, las variables estáticas se asignan a direcciones de memoria física, la pila de llamadas (call stack, runtime stack) se aloja a un bloque de memoria, así como el código mismo de la máquina.



Conceptos

Por cierto, ¿qué es la **pila de llamadas**?

Es una **estructura de memoria** de tipo **LIFO** (último que llega, primero que sale) encargada de las **llamadas a funciones**, la cuál es manejada por el CPU de manera **automática**.

- Cada vez que se llama una función se crea un marco por encima de la pila el cual se destruye una vez que la función regresa su valor.
- La pila almacena las variables locales, parámetros de la función, valores intermedios en los cálculos y la dirección de retorno (a dónde regresar después de evaluar la función).
- Solo el marco superior está activo. Las variables en otros marcos no son accesibles directamente.
- Permite el uso de funciones anidadas y recurrió.
- El CPU elimina un marco una vez que su función se ha ejecutado.



Conceptos

- **Tiempo de ejecución del programa:** Cuando el programa está corriendo, las variables se ligan a valores, como al ejecutar la asignación `edad = 18`.

En ocasiones un elemento puede ligarse en más de uno de estos tiempos. Por ejemplo: una variable puede ligarse a un valor durante el tiempo de compilación o de carga o al momento de ejecución. Aquí surge el concepto de enlace temprano y enlace tardío, dónde el primero se refiere a realizar el enlace tan rápido como sea posible y el segundo a generar la liga en la última oportunidad. El diseño debe considerar estas alternativas.



Conceptos

3. Fiabilidad

El programa debe funcionar de la misma manera cada vez que se ejecute con los mismos datos de entrada independientemente de que se encuentre en distintas plataformas.

Un aspecto importante en referencia a esto es la necesidad de diseñar en el lenguaje mecanismos para manejar los casos especiales.



Conceptos

4. Soporte

Debe ser sencillo aprender el lenguaje e instalarlo en la computadora propia de los usuarios. Idealmente, los compiladores deberían ser abiertos al público en vez de propiedad de una corporación y con altos costos. Además, el lenguaje debería estar implementado en múltiples plataformas.

La existencia de cursos, libros, tutoriales y canales de asistencia son importantes en el éxito del lenguaje.



Conceptos

5. Abstracción

La abstracción es un proceso fundamental. Los programadores pasan mucho tiempo construyendo abstracciones de datos y procedimientos para la reutilización del código y evitar reinventarlo.

Todo buen lenguaje de programación tiene un soporte tan bueno para la abstracción de datos y procedimientos que es la herramienta preferida en la mayoría de aplicaciones. En python, contamos con listas, diccionarios, conjuntos, etc.



6. Ortogonalidad

- Se dice que un lenguaje es ortogonal si las declaraciones y características se construyen a partir de un pequeño conjunto de operaciones primitivas que son mutuamente independientes.
- Cuando un programa es más ortogonal se requieren menos “casos especiales” para escribir programas correctos.
- Los programas tienden a ser más simples y claros.

Ejemplo: Considere pasar argumentos a una función. Un lenguaje totalmente ortogonal permitirá la entrada de cualquier tipo de datos, incluyendo la definición de una función.



Conceptos

- La mayoría de lenguajes imperativos no permiten pasar la definición de una función como argumento.
- Por otro lado, la no ortogonalidad frecuentemente se correlaciona con eficiencia ya que sus casos especiales eliminan opciones de programación las cuales consumen tiempo y/o espacio.



7. Implementación eficiente

- Las construcciones y las características de un lenguaje deben permitir implementaciones prácticas y eficientes en las plataformas contemporáneas.
- Las primeras implementaciones de **Java** sufrieron crítica por su desempeño en cuanto al tiempo de ejecución, pero esto ha cambiado con los nuevos sistemas de compilación de Java. Otros ejemplos con esta característica son: **Python, Ruby, PHP, Perl, Lua**.
- Algunos lenguajes que son difíciles para implementar código: lenguajes esotéricos, lenguajes de bajo nivel y lenguajes muy especializados: **Brainfuck** (ocho comandos), **Haskell** (lenguaje puramente funcional sin efectos secundarios)

Temática

- Conceptos básicos
- **Clasificación por niveles**
- Clasificación por paradigmas
- Paradigmas emergentes

Clasificación por niveles



Los lenguajes de programación se pueden clasificar de acuerdo al nivel de abstracción del hardware

- Bajo nivel
- Alto nivel



Niveles

Lenguajes de bajo nivel

- Cuentan con nulo o mínimo nivel de abstracción con respecto al conjunto de instrucciones que entiende el procesador.
- Se requiere un profundo entendimiento del hardware.

Lenguaje de máquina

Consiste enteramente de código binario (0's y 1's). Este lenguaje no requiere traducción; el CPU puede ejecutarlo directamente.

Lenguaje ensamblador

Emplea códigos legibles para el humano y fáciles de recordar que representan las instrucciones del computador (ADD, MOV, etc).

Requiere traducción al lenguaje de la computadora (ensamblador).



Niveles

Lenguaje de máquina

Se define a partir de la arquitectura del conjunto de instrucciones (ISA).

Ejemplos de ISA:

- **X86-64**: Se encuentra en la mayoría de computadoras personales y servidores. Cuenta con 16 registros de propósito general. Los registros tiene longitud de 64 bits. Memoria máxima de alrededor de 16 Exabytes. Es compatible con aplicaciones de 32 y 64 bits. Una arquitectura de 64 bits implica que el procesador puede procesar 64 bits en una sola operación.
- **X86**: Era el estándar anterior para computadoras personales. Se utiliza en sistemas embebidos dónde se tienen aplicaciones pequeñas con poco consumo de energía. Cuenta con 8 registros. Los registros tiene longitud de 32 bits. Memoria máxima de alrededor de 4 Gigabytes. No es compatible con aplicaciones de 64 bits.
- **ARM**: En las computadoras con procesadores Apple Silicon y en dispositivos móviles.



Arquitectura x86-64

Distintos nombres para la arquitectura

- AMD-64
- Intel64
- EM64T
- IA-32e



Niveles

Registros

Los registros son ubicaciones para almacenamiento dentro de la memoria del CPU mismo. En estos se almacenan datos temporales y direcciones.

Tenemos registros de propósito especial y de propósito general.

Registros de propósito especial:

- el **contador del programa (RIP o PC)**, el cual es un apuntador hacia la nueva instrucción que debe buscarse de memoria,
- el **apuntador de la pila (RSP)**, el cual apunta a la parte superior de la pila),
- Una vez se ha buscado la instrucción en el RIP, dicha instrucción se trae al CPU en el **registro de datos de memoria (MDR)**.
- La instrucción en ejecución se encuentra en el **registro de instrucciones (IR)**.



Niveles

Registros de propósito general

Estos almacenan operandos y datos temporales (intermedios).

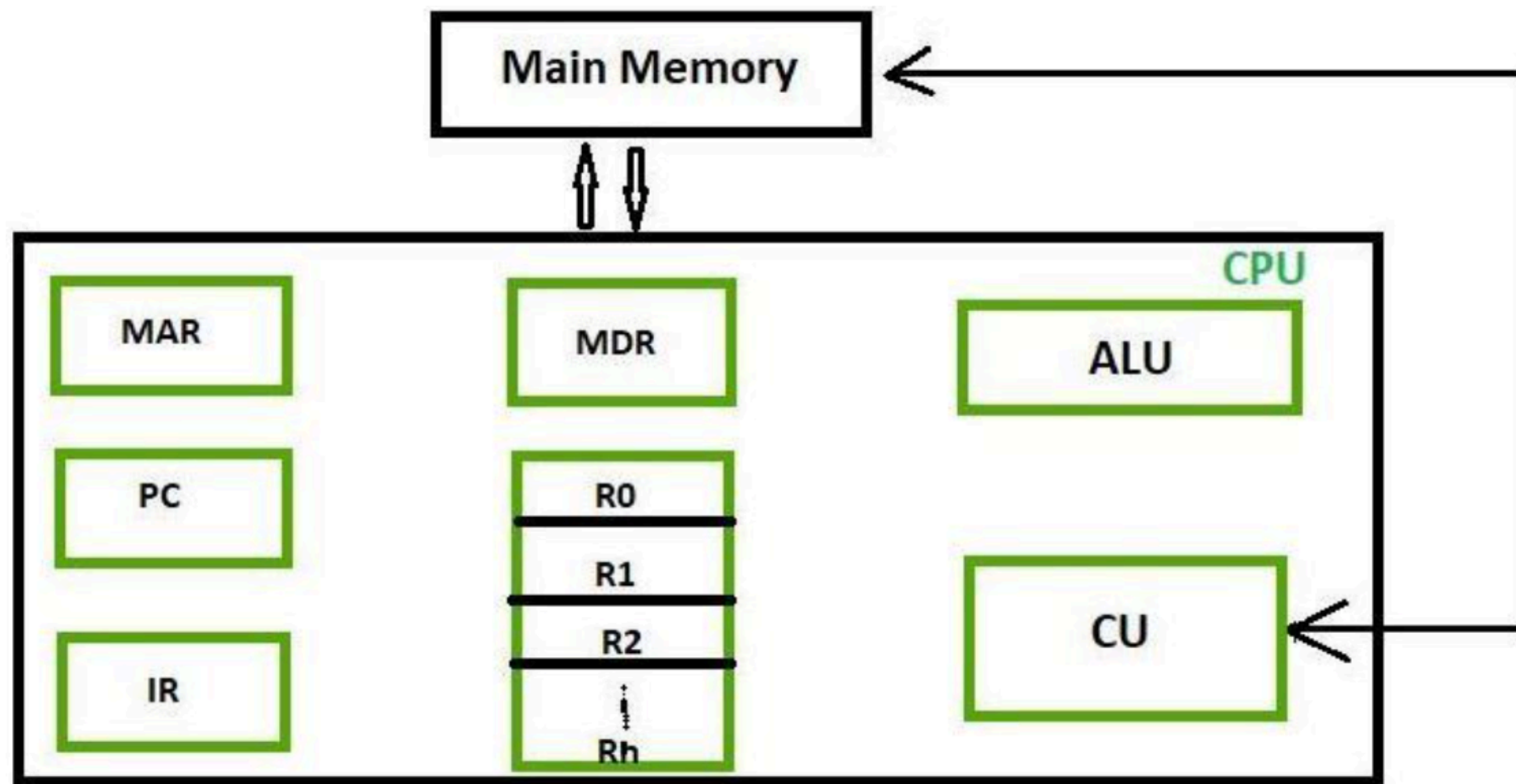
Register	Purpose
rax	Return values from functions
rbx	Base register (rarely used in x64)
rcx	Loop counters, shift counts
rdx	Data register (legacy use)
rsi	Source pointer for data transfer
rdi	Destination pointer for data transfer
rsp	Stack pointer
rbp	Base pointer for stack frames
r8-r15	Additional GPRs in x64

Fuente: <https://medium.com/@kyoungminroh01/x86-64-architecture-basics-8b6d9703a6e4>



Niveles

¿Cómo se ven los registros?

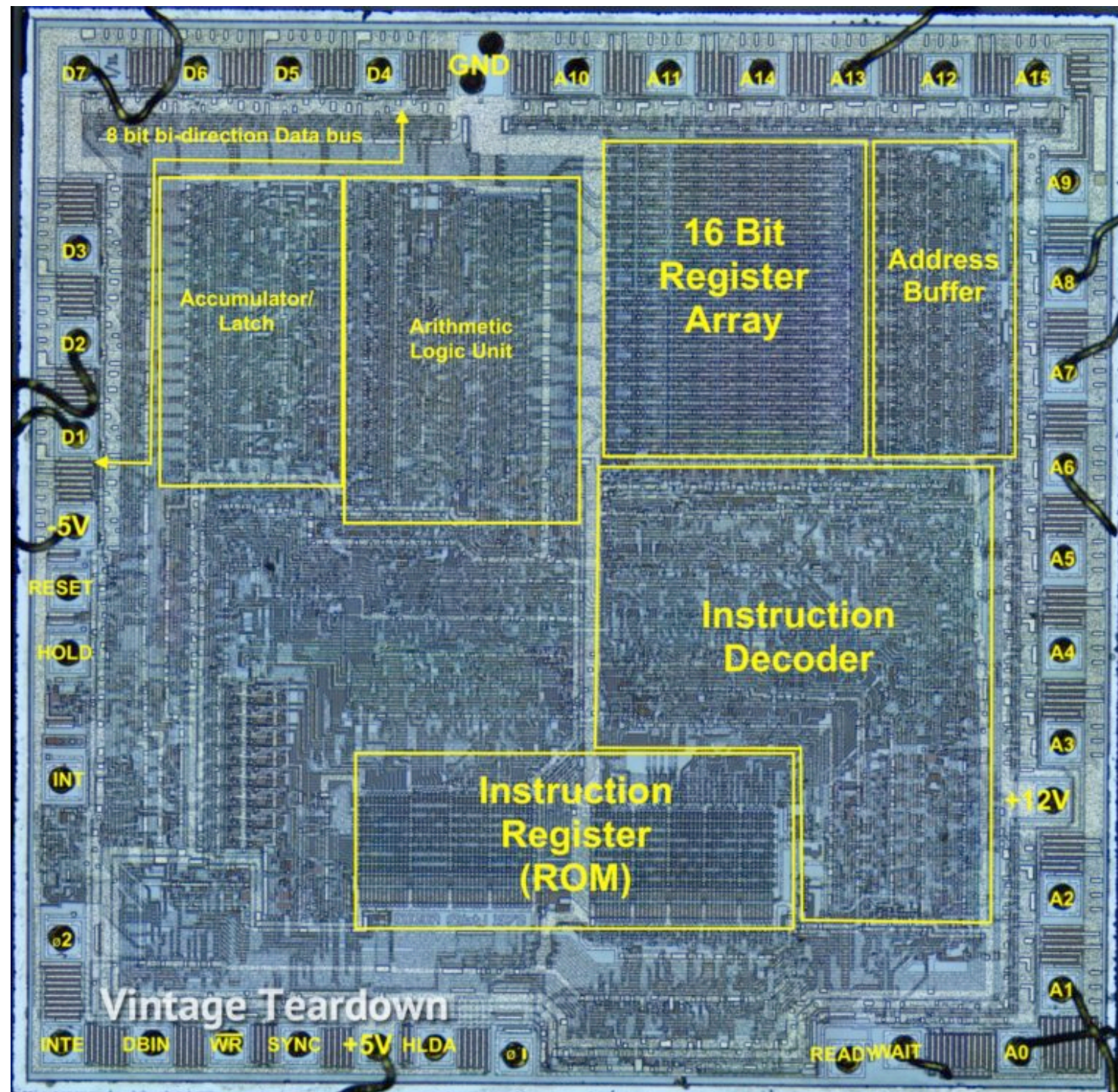


Fuente: <https://www.geeksforgeeks.org/computer-organization-architecture/different-classes-of-cpu-registers/>

Niveles



Una más



Intel 8080A



Niveles

Observa cómo se ve tu código de C++ en ensamblador y en binario en:

<https://godbolt.org/>



Niveles

Lenguajes de alto nivel

- Están diseñados para que el humano pueda escribirlos, leerlos y entenderlos fácilmente.
- Utilizan sintaxis similar al lenguaje natural.
- Tienen un alto nivel de abstracción; esconden los detalles complejos del hardware y del manejo de memoria.
- Deben ser traducidos por un compilador o intérprete a código de máquina.

Ejemplos: Java, JavaScript, Python, Julia, Ruby, PHP.



Niveles

En algunos escenarios las personas pueden utilizar una tercera clasificación, los **lenguajes de nivel intermedio**; estos ofrecen características de alto nivel tal como funciones, estructuras de datos y al mismo tiempo algunas características de bajo nivel para el control de la memoria y recursos del sistema. En esta clase tenemos tanto a **C** como a **C++**.

Manipular memoria dinámica

```
malloc(size)
calloc(num, size)
realloc(*ptr, size)
free(*ptr)
```

Funciones memoria en general
(ubicadas en <string.h>)

```
memcpy(*dest, *src, n)
memmv(*dest, *src, n)
memset(*ptr, value, n)
memcmp(*ptr_1, *ptr_2, n)
memchr(*ptr, value, n)
```

Ver ejemplo en códigos del Profesor (**example_memory.c**)

Temática

- Conceptos básicos
- Clasificación por niveles
- **Clasificación por paradigmas**
- Paradigmas emergentes

Clasificación por paradigmas



Conjunto de metodologías y conceptos que nos guían como razonamos e implementamos el código.

- Cada paradigma ofrece una perspectiva distinta sobre como estructurar y resolver problemas computacionales.
- Estas guías nos guían no sólo a como realizar las tareas sino a como manipular la información.
- Muchos de los lenguajes modernos están diseñados para poder emplear múltiples paradigmas, es decir, soportan distintos enfoques para resolver los problemas.
- Podemos tomar los fuertes de distintos paradigmas al mismo tiempo.



Paradigmas

¿Porqué son importantes?

- Mejoramos nuestras habilidades para razonar y resolver problemas de la manera más sencilla.
- Código de mejor calidad: se puede lograr un código más mantenible, leíble y eficiente al tener conocimiento los distintos enfoques.
- Adaptabilidad: la informática es un campo en constante evolución. Al tener conocimiento de los paradigmas estaremos preparados para las nuevas metodologías que estarán surgiendo en el futuro.



Paradigmas

¿Porqué son importantes?

- Colaboración: En la jerga de los profesionales de la computación las personas se refieren a los paradigmas empleados en su código. El conocerlos nos permite entender mejor el problema y tener mejor colaboración.
- Empleos: el conocer los distintos enfoques es una señal de un conocimiento más profundo de las ciencias computacionales y es un punto de partida para algunos empleadores.